PROJECT SYNOPSIS
On

TEXT-BASED ADVENTURE GAME

RU-100-01-00021

Programming Fundamentals and Logic Building Using Python

SESSION: 2026–27

Submitted By:

Shoaib Akhtar

RU ID: RU-26-00647

Bachelor of Technology in Computer Science & Engineering with AI and ML in association with IBM

Under the Guidance of:

Mr. Priyanshu

(IBM SME and EXPERT)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

Contents

1. Introduction and Problem Statement

2. Objectives

3. Concepts and Requirements Mapping

4. Program Design

5. Implementation

6. Testing

7. Sample Runs

8. Challenges and Solutions

9. Future Enhancements

10. Conclusion

11. References

1. Introduction and Problem Statement

Text-based adventure games were among the earliest computer games. The player reads descriptions and makes decisions by typing choices, and the program responds to each decision. Building one is a good first project because it needs only the core ideas of programming: taking input, making decisions, repeating actions and organising code into functions.

Problem statement. Create a text-based adventure game in which the player explores a forest, a cave and a river by choosing options from a menu, faces a random monster attack, collects score and ends the game by quitting or losing all health.

The game rules given in the assignment are:

- The player starts in the Forest with Health 100 and Score 0.
- Menu choices: 1 Cave, 2 River, 3 Quit.
- Cave: a monster attacks with random damage from 10 to 40 (fixed to 30 for testing). If the player survives, a key gives +20 score.
- River: a treasure gives +50 score.
- The game ends when the player quits or Health reaches 0.

2. Objectives

- Understand the problem and define it clearly.
- Design the solution with functions, conditions and a loop.
- Use the random module for monster damage and track Health and Score.
- Validate every menu choice so wrong input never breaks the game.
- Test the program with the four given test cases and additional edge cases.
- Document the work and publish the code and report on GitHub.

3. Concepts and Requirements Mapping

3.1 Programming concepts used

Concept	Where it is used	Purpose
Conditions	if / elif / else in play_game()	Select the action and check whether the player survived.
Loops	while health > 0 in play_game(); while True in show_menu()	Repeat the game and ask again after invalid input.
Functions	show_menu(), cave(), river(), play_game()	Keep menu input, each place, and the main game loop in clear functions.
Random	random.randint(10, 40) in cave()	Produce unpredictable monster damage.

3.2 Bare minimum requirements checklist

Requirement	Implementation	Status
Python 3 solution in one .py file, run from the console	adventure_game.py	Done
Functions for each place; conditions for choices	cave(), river(); if / elif / else	Done
Loop until the player quits or Health reaches 0	while health > 0 in play_game()	Done
random module for damage; track Health and Score	random.randint() in cave(); score and health variables	Done
Validate every menu choice; show final score at the end	Validation loop in show_menu(); final score on quit and Game Over	Done

4. Program Design

4.1 Algorithm

1. Start. Set Health = 100 and Score = 0.
2. While Health is greater than 0, show the Forest menu and read the choice.
3. If the choice is not 1, 2 or 3, print an error message and ask again. Health and Score do not change.
4. If the choice is 1 (Cave): generate the damage, reduce Health (never below 0) and print the attack. If Health is still above 0, add 20 to Score and print that a key was found.
5. If the choice is 2 (River): add 50 to Score and print the treasure message with the current Score and Health.
6. If the choice is 3 (Quit): print the final Score and Health and stop.
7. If Health reaches 0, the loop ends: print Game Over with the final Score and stop.

4.2 Flowchart

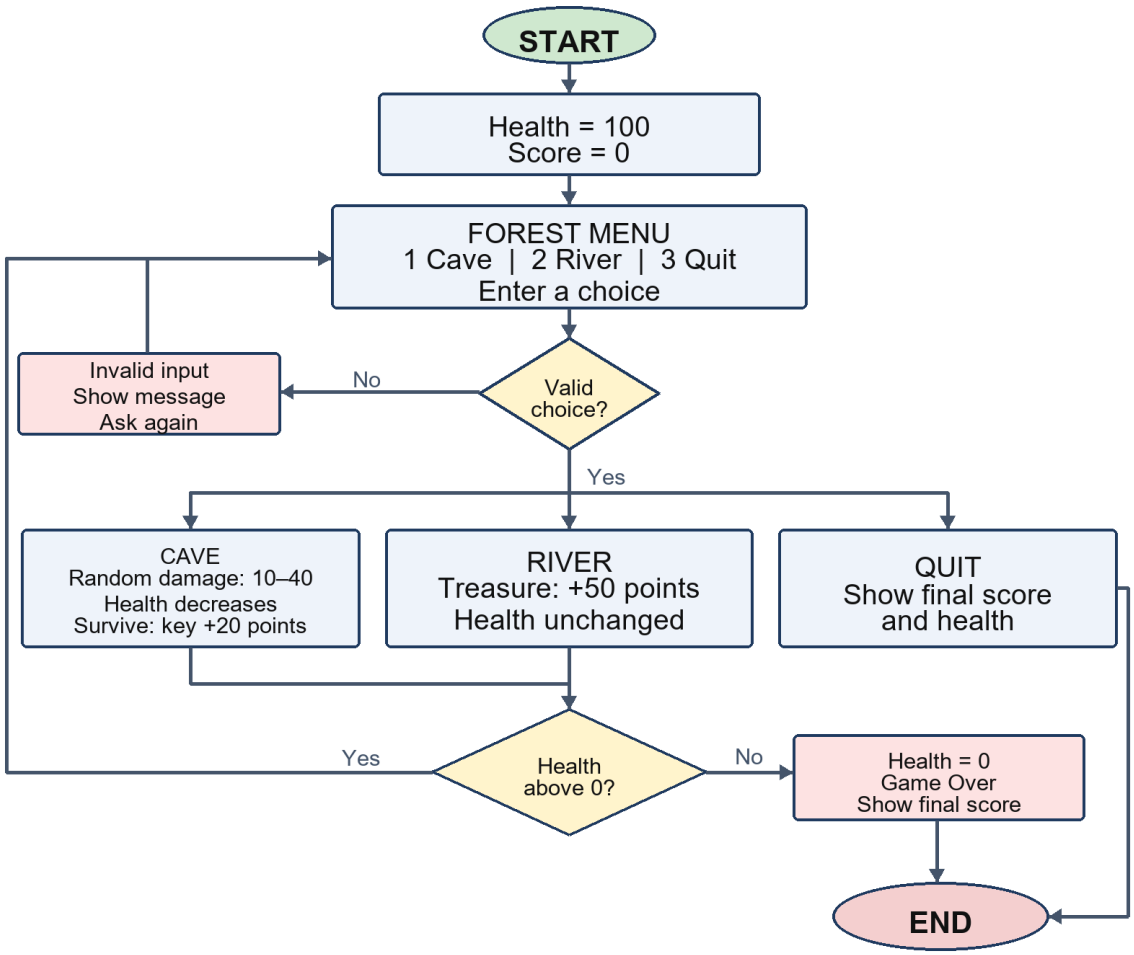


Figure 1: Flowchart of the Text-Based Adventure Game

4.3 Functions and program entry point

Function or section	Description
show_menu()	Prints the menu and repeats until the player enters 1, 2, or 3.
cave(score, health, fixed_damage=None)	Applies monster damage and awards key points if the player survives.
river(score, health)	Awards treasure points and leaves health unchanged.
play_game(starting_health, fixed_damage)	Runs the game loop, updates score and health, prints the ending, and returns final values.
if __name__ == '__main__'	Starts play_game() with default settings when the file is run directly.

4.4 Constants and variables

Name	Value	Meaning
STARTING_HEALTH	100	Starting health of the player

Name	Value	Meaning
MIN_DAMAGE / MAX_DAMAGE	10 / 40	Range of random monster damage
KEY_POINTS	20	Points for surviving a Cave attack
TREASURE_POINTS	50	Points for finding treasure at the River
score, health	0, 100	Player state, updated during the game
fixed_damage	None by default	Optional value used to make an example predictable

5. Implementation

The complete program is a single file, adventure_game.py. It uses the built-in random module for monster damage and separate functions for the menu, Cave, River, and main game loop.

5.1 Menu input validation

The choice is read as text and compared with the strings "1", "2" and "3". This avoids converting the input with int(), which would crash on text such as abc. Any other input, including an empty line, prints the error message and the loop asks again.

```
def show_menu():
    print("\nYou are in the Forest. Where do you want to go?")
    print("1. Cave")
    print("2. River")
    print("3. Quit")

    while True:
        choice = input("Choice: ").strip()
        if choice in ("1", "2", "3"):
            return choice
        print("Invalid choice. Enter 1, 2 or 3.")
```

5.2 Game choices

The cave() function reduces health with max(0, health - damage), so it can never become negative. It awards the key only if the player survives. The river() function awards the treasure without changing health.

```
def cave(score, health, fixed_damage=None):
    if fixed_damage is None:
        damage = random.randint(MIN_DAMAGE, MAX_DAMAGE)
    else:
        damage = fixed_damage

    health = max(0, health - damage)
    print(f"Monster attacks! Damage: {damage}. Health: {health}")

    if health > 0:
        score += KEY_POINTS
        print(f"You found a key! Score: {score}")

    return score, health


def river(score, health):
    score += TREASURE_POINTS
    print("You found a treasure!")
    print(f"Score: {score} | Health: {health}")
    return score, health
```

5.3 Main game loop

The while loop keeps the game running while the player is alive. The if / elif / else chain calls cave(), river(), or ends the game. The final score is displayed when the player quits and when the game is over.

```
def play_game(starting_health=STARTING_HEALTH, fixed_damage=None):
    health = starting_health
    score = 0

    while health > 0:
        choice = show_menu()

        if choice == "1":
            score, health = cave(score, health, fixed_damage)
        elif choice == "2":
            score, health = river(score, health)
        else:
            print(f"You left the adventure. Final score: {score} | Health: {health}")
            return score, health

    print(f"Game Over! Final score: {score}")
    return score, health


if __name__ == "__main__":
    play_game()
```

5.4 Random damage and predictable examples

The cave() function uses random.randint(10, 40) for monster damage. For predictable examples, play_game accepts optional starting_health and fixed_damage values. The normal game uses the defaults when the file is run.

```
python adventure_game.py

# Predictable example from Python:
from adventure_game import play_game
play_game(starting_health=30, fixed_damage=30)
```

6. Testing

The project includes four main test cases and additional unit tests in tests/test_adventure_game.py. The tests use Python's unittest module and can be run from the project folder with python -m unittest discover -s tests -v.

6.1 Test cases from the problem statement

No.	**Input**	**Expected output**	Test assertion	Expected result
1 Normal	Choice: 2 (River)	You found a treasure! Score: 50 | Health: 100	Covered by test	Expected
2 Boundary	Starting Health 30; Choice 1 (Cave); damage fixed to 30	Monster attacks! Damage: 30. Health: 0 Game Over! Final score: 0	Covered by test	Expected

No.	Input	Expected output	Test assertion	Expected result
3 Invalid	(a) Choice: 7 (b) Choice: abc	Invalid choice. Enter 1, 2 or 3. (for each) No health lost	Covered by test	Expected
4 Valid	Damage fixed to 30; Choices 1, 2, 3	Health 70; key, Score 20; treasure, Score 70; final score 70, Health 70	Covered by test	Expected

6.2 Additional edge-case tests

Scenario	Result
Random monster damage	Always between 10 and 40
Damage larger than remaining health	Health stops at 0, never negative
Health 31 and damage 30 (survive with 1 health)	Key is still found; Score 20
Player dies in the cave	No key; score earned earlier is kept; loop stops without asking for more input
Inputs: empty line, space, 0, 4, -1, 1.5	All rejected with the error message; health unchanged
Input with spaces around a valid choice (" 2 ")	Accepted as choice 2
Fixed damage passed to cave() or play_game()	Returns 30 for a predictable example
Randomised game states	Health always within 0 to 100; score always a multiple of 10

6.3 Console output of the test cases

Test Case 1: Normal input

```
========================================
       TEXT-BASED ADVENTURE GAME
========================================
You start in the Forest. Health: 100 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 2
You found a treasure!
Score: 50 | Health: 100

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 3
You left the adventure. Final score: 50 | Health: 100
```

Test Case 2: Boundary input (starting_health=30, fixed_damage=30)

```
========================================
       TEXT-BASED ADVENTURE GAME
========================================
You start in the Forest. Health: 30 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 30. Health: 0
Game Over! Final score: 0
```

Test Case 3: Invalid input

```
========================================
       TEXT-BASED ADVENTURE GAME
========================================
You start in the Forest. Health: 100 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 7
Invalid choice. Enter 1, 2 or 3.
Choice: abc
Invalid choice. Enter 1, 2 or 3.
Choice: 3
You left the adventure. Final score: 0 | Health: 100
```

Test Case 4: Mixed choices (fixed_damage=30)

```
========================================
       TEXT-BASED ADVENTURE GAME
========================================
You start in the Forest. Health: 100 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 30. Health: 70
You found a key! Score: 20

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 2
You found a treasure!
Score: 70 | Health: 70

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 3
You left the adventure. Final score: 70 | Health: 70
```

7. Sample Runs

Two example games with random monster damage are shown below. One ends when the player quits; the other ends when the player's health reaches zero.

Run A: player quits

```
========================================
       TEXT-BASED ADVENTURE GAME
========================================
You start in the Forest. Health: 100 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 20. Health: 80
You found a key! Score: 20

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 2
You found a treasure!
Score: 70 | Health: 80

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 40. Health: 40
You found a key! Score: 90

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 14. Health: 26
You found a key! Score: 110

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 3
You left the adventure. Final score: 110 | Health: 26
```

Run B: Game Over

```
======================================
       TEXT-BASED ADVENTURE GAME
======================================
You start in the Forest. Health: 100 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 2
You found a treasure!
Score: 50 | Health: 100

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 17. Health: 83
You found a key! Score: 70

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 28. Health: 55
You found a key! Score: 90

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 27. Health: 28
You found a key! Score: 110

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 14. Health: 14
You found a key! Score: 130

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 21. Health: 0
Game Over! Final score: 130
```

8. Challenges and Solutions

Challenge	Solution
Wrong input such as abc would crash the program if converted with int()	Read the input as a string and compare it with "1", "2" and "3" inside a loop.
Random damage makes the test cases impossible to repeat	Added the optional fixed_damage function parameter so examples can use a known damage value.
Health could become negative after a strong attack	Used max(0, health - damage) so that health stops at 0.

Challenge	Solution
The key must be given only if the player survives	Checked health > 0 after the attack before adding the score.
Checking the output by eye is slow and error-prone	Wrote automated unittest tests that feed scripted input and check the printed output.

9. Future Enhancements

- High-score table saved to a file.
- More places, items and an inventory, for example a healing potion.
- Difficulty levels that change monster damage.
- Coloured console output and, later, a graphical interface.

10. Conclusion

The Text-Based Adventure Game meets the core requirements of PS No. 39. It uses functions, conditions, a loop, input validation, and the random module for monster damage. The player can quit or the game ends when health reaches zero. The project helped me practise breaking a problem into smaller functions and tracking health and score as the game runs.

11. References

1. Python Software Foundation. random: Generate pseudo-random numbers. https://docs.python.org/3/library/random.html

1.

2. Python Software Foundation. unittest: Unit testing framework. https://docs.python.org/3/library/unittest.html

3. Python Software Foundation. The Python Tutorial. https://docs.python.org/3/tutorial/

4. Graphviz. Graph visualization software, used for the flowchart. https://graphviz.org